Software Documentation Done Right

READMEs, API docs, comments, tools and best practices

Plan • Write • Publish • Maintain

Updated October 2026

Contents

1. Why documentation matters
2. The four types of documentation
3. Writing a great README
4. Code comments and docstrings
5. API documentation
6. Architecture and decision records
7. Docs-as-code and tools
8. Writing style and structure
9. Maintenance and checklist

> **Core idea**
> Documentation is part of the product. Code that nobody can understand, install or use might as well not exist. Good docs save time for your users, your teammates and your future self.

Who needs documentation?

- **Users** who want to install and use your software.
- **Developers** who integrate with your API or library.
- **Contributors and new teammates** who must understand how the code works.
- **Operators** who deploy, monitor and fix it in production.

1. Why documentation matters

Teams often treat documentation as an afterthought. The costs show up later in slow onboarding, repeated questions, support tickets and fragile knowledge held in one person's head.

Benefit	What it looks like in practice
Faster onboarding	A new developer runs the project on day one without asking for help.
Fewer interruptions	Answers are written once and linked, instead of explained repeatedly.
Better adoption	Users and API consumers succeed quickly and keep using the product.
Safer changes	Documented decisions explain why the code works the way it does.
Lower bus factor	Knowledge is shared, so the team can survive people leaving or going on holiday.

Common documentation problems

- Docs are **missing**: nobody had time to write them.
- Docs are **outdated**: the code changed but the docs did not.
- Docs are **hard to find**: scattered across wikis, chats and personal notes.
- Docs are **hard to read**: too long, too vague or written for the author, not the reader.

> **Rule of thumb**
> Write the docs you wished existed when you started. If you had to ask a question to get unstuck, the answer belongs in the documentation.

2. The four types of documentation

A popular framework called **Diataxis** divides documentation into four kinds. Each serves a different need, and mixing them in one page makes docs confusing.

Type	Purpose	Reader is...	Example
Tutorial	Teach by guiding a beginner through a complete, working example	Learning	"Build your first app in 15 minutes"
How-to guide	Solve a specific, real-world task	Working	"How to deploy to production"
Reference	Describe facts precisely and completely	Looking something up	API endpoints, configuration options, CLI flags
Explanation	Give background, reasoning and context	Understanding	"Why we use event sourcing"

How to apply it

- **Tutorials** must work end to end. Keep them short, concrete and free of side discussions.
- **How-to guides** assume some knowledge and focus on steps to reach a goal.
- **Reference** should be accurate and consistent. Generate it from code where possible.
- **Explanations** can discuss alternatives and trade-offs. They do not contain step-by-step instructions.

Other useful documents

Document	Purpose
README	The front door: what the project is and how to start
CONTRIBUTING	How to propose changes, coding standards, review process
CHANGELOG	What changed in each release
LICENSE	The terms under which others may use the code
SECURITY	How to report vulnerabilities responsibly
Runbook	Steps for operating the system and handling incidents

3. Writing a great README

The README is the first file people read. It should answer three questions in under a minute: **What is this? Why should I care? How do I start?**

A reliable template (Markdown)

````
# Project Name

One sentence describing what it does and who it is for.

![Build status](badge-url)  ![License](badge-url)

## Features
- Key feature one
- Key feature two

## Quick start
```bash
git clone https://github.com/you/project.git
cd project
npm install
npm start
```

## Usage
Show the most common example with real code.

## Configuration
| Variable   | Description          | Default |
|------------|----------------------|---------|
| PORT       | Port to listen on    | 3000    |

## Documentation
Link to the full docs.

## Contributing
See CONTRIBUTING.md.

## License
MIT
````

README tips

- Put the **quick start** near the top and make every command copy-paste ready.
- Include a **screenshot, GIF or diagram** for user interfaces.
- State **prerequisites** and supported versions clearly.
- Test your instructions on a clean machine; they often hide missing steps.

4. Code comments and docstrings

Comments explain code to other developers. Use them to describe **why**, not **what**. The code itself should show what it does.

Weak vs helpful comments

```
// Bad: repeats the code
i = i + 1;   // add one to i

// Good: explains the reason
// The API rate-limits to 100 requests per minute, so we pause here.
await Task.Delay(600);
```

What to comment

- Business rules and non-obvious decisions
- Workarounds for bugs in other systems (with a link to the issue)
- Performance trade-offs and known limitations
- TODO notes, with a ticket number or owner

Docstrings and doc comments

These are structured comments attached to functions, classes and modules. Tools turn them into reference documentation and show them in editor tooltips.

Language	Style	Doc generator
Python	Docstrings (Google, NumPy or reST style)	Sphinx, MkDocs with mkdocstrings
JavaScript / TypeScript	JSDoc / TSDoc comments	TypeDoc, JSDoc
C# / .NET	XML documentation comments (`///`)	DocFX, Sandcastle
Java	Javadoc	Javadoc tool
Go / Rust	Doc comments	`go doc`, `cargo doc`

```
def convert(amount: float, rate: float) -> float:
    """Convert an amount using an exchange rate.

    Args:
        amount: Value in the source currency.
        rate: Units of target currency per unit of source currency.

    Returns:
        The converted value, rounded to 2 decimal places.

    Raises:
        ValueError: If rate is not positive.
    """
    if rate <= 0:
        raise ValueError("rate must be positive")
    return round(amount * rate, 2)
```

5. API documentation

Developers judge an API by its documentation. Great API docs let someone make a successful request in minutes.

What good API docs include

- **Getting started:** how to get credentials and make a first call.
- **Authentication:** methods, header formats, token lifetimes.
- **Endpoint reference:** URL, method, parameters, request body, responses.
- **Examples:** working requests in curl and popular languages.
- **Errors:** status codes, error formats and how to fix each.
- **Limits and versioning:** rate limits, pagination, deprecation policy.
- **Changelog:** what changed between versions.

OpenAPI: a standard description format

OpenAPI (formerly Swagger) describes REST APIs in a YAML or JSON file. Tools can generate interactive docs, client libraries and tests from it.

```
openapi: 3.0.3
info:
  title: Todo API
  version: 1.0.0
paths:
  /todos/{id}:
    get:
      summary: Get a todo by ID
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer }
      responses:
        "200":
          description: The todo item
        "404":
          description: Todo not found
```

Tool	Use
Swagger UI / Redoc	Render an OpenAPI file as interactive documentation
Postman collections	Share runnable example requests
Stoplight, Scalar	Design and publish API docs

6. Architecture and decision records

Beyond how to use the code, teams need to know how the system is structured and **why** it was built that way.

Architecture overview

- A one-page **system diagram** showing major components and how they communicate.
- The **C4 model** offers four zoom levels: context, containers, components and code.
- Data flow, deployment layout and external dependencies.
- Text-based diagram tools such as **Mermaid** and **PlantUML** keep diagrams in Git next to the code.

Architecture Decision Records (ADRs)

An **ADR** is a short document capturing one important decision. Store them in a folder such as `docs/adr/`, numbered in order, and never delete old ones; mark them superseded instead.

```
# ADR 007: Use PostgreSQL for the primary database

Status:  Accepted (2026-03-14)

## Context
We need transactional integrity and complex queries.
Options considered: PostgreSQL, MySQL, MongoDB.

## Decision
Use PostgreSQL 16, hosted as a managed service.

## Consequences
+ Strong consistency and a rich SQL feature set.
+ Team already has experience with it.
- We must manage schema migrations carefully.
```

Why ADRs help
Six months later, nobody remembers why MongoDB was rejected. An ADR answers that in two minutes and prevents the same debate from restarting.

7. Docs-as-code and tools

Docs-as-code means treating documentation like source code: written in plain text (usually Markdown), stored in Git, reviewed in pull requests, and published automatically by CI.

Benefits

- Docs live next to the code, so they are updated in the **same pull request**.
- Version history shows who changed what and why.
- Reviews and automated checks (spelling, broken links) keep quality high.

Documentation tools

Tool	Best for	Format
MkDocs (Material)	Clean project documentation sites, easy to start	Markdown
Docusaurus	Docs with versioning and a blog, React-based	Markdown / MDX
Sphinx	Python projects and large technical manuals	reStructuredText / Markdown
Read the Docs	Free hosting and builds for open-source docs	Sphinx or MkDocs
GitHub Pages	Free static site hosting from a repository	Static HTML
GitBook, Notion, Confluence	Team knowledge bases with a visual editor	Rich text
DocFX	API docs for .NET projects	Markdown + XML comments

Quick start with MkDocs

```
pip install mkdocs-material
mkdocs new my-docs
cd my-docs
mkdocs serve        # live preview at http://127.0.0.1:8000
mkdocs build        # generate the static site
```

Automated checks to add to CI

- **Broken link checkers** (for example lychee or markdown-link-check)
- **Markdown linters** (markdownlint) and **prose linters** (Vale)
- **Spell checkers** and a build step that fails on warnings

8. Writing style and structure

Good technical writing is clear, direct and organized around the reader's goal.

Style guidelines

- **Know your audience.** Beginner, expert or operator? State prerequisites.
- **Lead with the answer.** Put the most important information first.
- **Use short sentences** and plain words. Avoid jargon, or define it on first use.
- **Use the active voice and imperative steps:** "Click Save", not "The Save button should be clicked".
- **Be consistent.** Use one term for one concept throughout.
- **Show, do not just tell.** Provide runnable examples and expected output.

Structure guidelines

- Use **descriptive headings** so readers can scan.
- Number steps in procedures; use bullets for unordered lists.
- One page, one purpose; link to related topics instead of repeating them.
- Use tables for comparisons and configuration options.
- Put code in formatted blocks with the language marked.

Before and after

Weak	Better
"It may be possible to configure the timeout parameter by modification of the settings file."	"Set `timeout` in `config.yml` to change the request timeout."
"Simply run the build."	"Run `npm run build`. The output appears in `dist/`."
"Click the thing at the top right."	"Click **Settings** in the top-right corner."

> **Avoid "simply" and "just"**
> Words like **simply**, **just** and **obviously** make readers who are struggling feel bad. Remove them.

9. Maintenance and checklist

Documentation that is not maintained becomes misleading. Build upkeep into your normal workflow.

Keep docs fresh

- Add a **docs update** item to your pull request template.
- Treat a code change without a docs change as **incomplete** when behavior changes.
- Review documentation on a schedule (for example every quarter).
- Mark each page with a **last reviewed** date and an owner.
- Add a **"Was this helpful?"** link or an "Edit this page" button to collect feedback.
- Watch support questions: repeated questions point to missing docs.

Versioning and changelogs

Use **Semantic Versioning** (MAJOR.MINOR.PATCH): increase MAJOR for breaking changes, MINOR for new backward-compatible features, PATCH for fixes. Keep a **CHANGELOG** grouped by release with sections such as Added, Changed, Deprecated, Removed, Fixed and Security.

```
## [1.4.0] - 2026-09-30
### Added
- Export to CSV from the reports page.
### Fixed
- Crash when the config file is empty.
```

Documentation checklist

- [ ] README explains what, why and how to start
- [ ] Quick start works on a clean machine
- [ ] Installation and configuration are documented
- [ ] Public APIs have reference docs and examples
- [ ] Errors and troubleshooting steps are listed
- [ ] Architecture overview and key decisions (ADRs) exist
- [ ] CONTRIBUTING, LICENSE and SECURITY files are present
- [ ] Docs are in Git and published automatically
- [ ] Links and code samples are tested in CI
- [ ] An owner and a review schedule are assigned

Final advice
Start small. A clear README and a working quick start beat a huge wiki nobody reads. Improve a little with every pull request.